To seriously speed up things, however, what we really need is distributing even the sharding step, with a new MapReduce job. We know that each point can be compared to cells independently; therefore, if we split the dataset in random shards, each shard can be processed by a battery of mappers. Reducers, at the same time, will just group points by (extended) cell(s)[22], and finally produce (not randomly, this time) new shards that can then be used in the first step of the MR-DBSCAN job (the one illustrated in figure 13.13). As a further optimization, since reducers for this job (sharding) will already have all the data for a cell, we can re-purpose the same machines to be the mappers in the MR-DBSCAN job.

It's also worth mentioning that the number of adjacent cells grows linearly with the dimension of the space, since the number of faces of a `d`-dimensional hypercube is equal to `2*d`: this means that the sharding algorithm can scale out to higher dimensions.

Finally, it's worth mentioning that, since we are performing this sharding step as an independent MapReduce job, we are forced to use a regular grid: on the contrary, the article presenting *He et al.* presenting MR-DBSCAN uses a different, more sophisticated approach, where statistics on the dataset are collected in a first pass through the dataset, splitting the domain into an irregular collection of parallel rectangles (see figure 13.14) each with (ideally) uniform density; then, leveraging the statistics collected, we can also tune the parameters for DBSCAN to adapt to the different density found in different cells.